Ecossistema Científico em Python: NumPy, Matplotlib e Pandas

Samir A. M. Martins[1]

[1]Universidade Federal de São João del-Rei
Departamento de Engenharia Elétrica
São João del-Rei, MG – Brasil

Graduação em Engenharia Elétrica

Nossa Missão de Hoje

Organizar dados,

calcular, visualizar

e interpretar.

Em Python, esse fluxo aparece naturalmente com **NumPy** para cálculo, **Matplotlib** para gráficos e **Pandas** para tabelas e análise de dados.

A Receita da Aula

Vamos seguir uma sequência lógica, muito parecida com um fluxo real de trabalho:

1. **Representar os dados** em arrays e tabelas.
2. **Fazer operações** numéricas de forma vetorizada.
3. **Visualizar padrões** com gráficos.
4. **Resumir informações** com seleção, filtros e agregações.

Ideia central

As três bibliotecas não competem entre si. Elas se complementam.

Visão Geral

Quem faz o quê?

NumPy Estruturas numéricas rápidas baseadas em arrays multidimensionais.

Matplotlib Ferramenta de visualização para transformar dados em gráficos.

Pandas Estruturas tabulares com rótulos, seleção flexível e operações de análise.

Uma interpretação prática

Pense em NumPy como a calculadora vetorial, Matplotlib como a prancheta de desenho e Pandas como a planilha programável.

Importando as Ferramentas

Código de preparação

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

- `np`, `plt` e `pd` são os apelidos padrão da comunidade.
- Usar a convenção ajuda a ler exemplos, livros e documentações.
- Nos arquivos de código da aula, esses apelidos serão mantidos.

NumPy

Por que começar com NumPy?

- Muitos dados numéricos em Python são organizados como arrays.
- Operações vetorizadas evitam laços explícitos desnecessários.
- O NumPy serve de base para várias bibliotecas de ciência de dados.

Criando Arrays e Sequências

Exemplos básicos

```
notas = np.array([6.5, 7.0, 8.5, 9.0])
tempo = np.linspace(0, 10, 6)
pares = np.arange(0, 12, 2)

print(notas)
print(tempo)
print(pares)
```

- `np.array()` cria arrays a partir de listas.
- `linspace` divide um intervalo em pontos igualmente espaçados.
- `arange` cria progressões com passo definido.

Shape, Tipo e Tamanho

Inspecionando um array

```
medidas = np.array([[1.2, 1.5, 1.7],
                    [2.0, 2.2, 2.4]])

print(medidas.shape)
print(medidas.dtype)
print(medidas.size)
```

Leitura rápida

`shape` informa a dimensão, `dtype` mostra o tipo e `size` conta quantos elementos existem ao todo.

Indexação e Fatiamento

Acessando partes do array

```
M = np.array([[10, 11, 12],
              [20, 21, 22],
              [30, 31, 32]])

print(M[0, 1])   # elemento da linha 0, coluna 1
print(M[:, 2])   # terceira coluna
print(M[1:, :2]) # submatriz
```

Ideia

A mesma sintaxe de slicing de listas continua útil, mas agora vale para várias dimensões.

Operações Vetorizadas

Sem laço explícito

```
temperaturas_c = np.array([18, 21, 25, 27])
temperaturas_f = temperaturas_c * 9 / 5 + 32

print(temperaturas_f)
```

- Soma, subtração, multiplicação e divisão ocorrem elemento a elemento.
- A escrita fica mais compacta e mais próxima da notação matemática.

Funções Úteis do NumPy

Resumo estatístico rápido

```
dados = np.array([4, 6, 8, 10, 12])

print(np.sum(dados))
print(np.mean(dados))
print(np.max(dados))
print(np.min(dados))
```

- Essas funções aparecem com frequência em pré-processamento e análise exploratória.
- Também funcionam em arrays bidimensionais, inclusive por eixo.

Matrizes e Álgebra Linear

Multiplicação matricial e sistema linear

```
A = np.array([[3, 1],
              [1, 2]])
B = np.array([[2, 0],
              [1, 4]])
b = np.array([9, 8])

print(A @ B)
print(np.linalg.solve(A, b))
```

Cuidado

O operador `*` faz multiplicação elemento a elemento. Para multiplicação matricial, use `@` ou `np.dot()`.

Matplotlib

Por que o Matplotlib entra depois?

Fluxo natural

Primeiro organizamos os números. Depois desenhamos aquilo que queremos comunicar.

- O Matplotlib trabalha muito bem com arrays NumPy.
- Quase todo gráfico segue três passos: dados, desenho e exibição/salvamento.

O Gráfico de Linha Mais Simples

Código mínimo

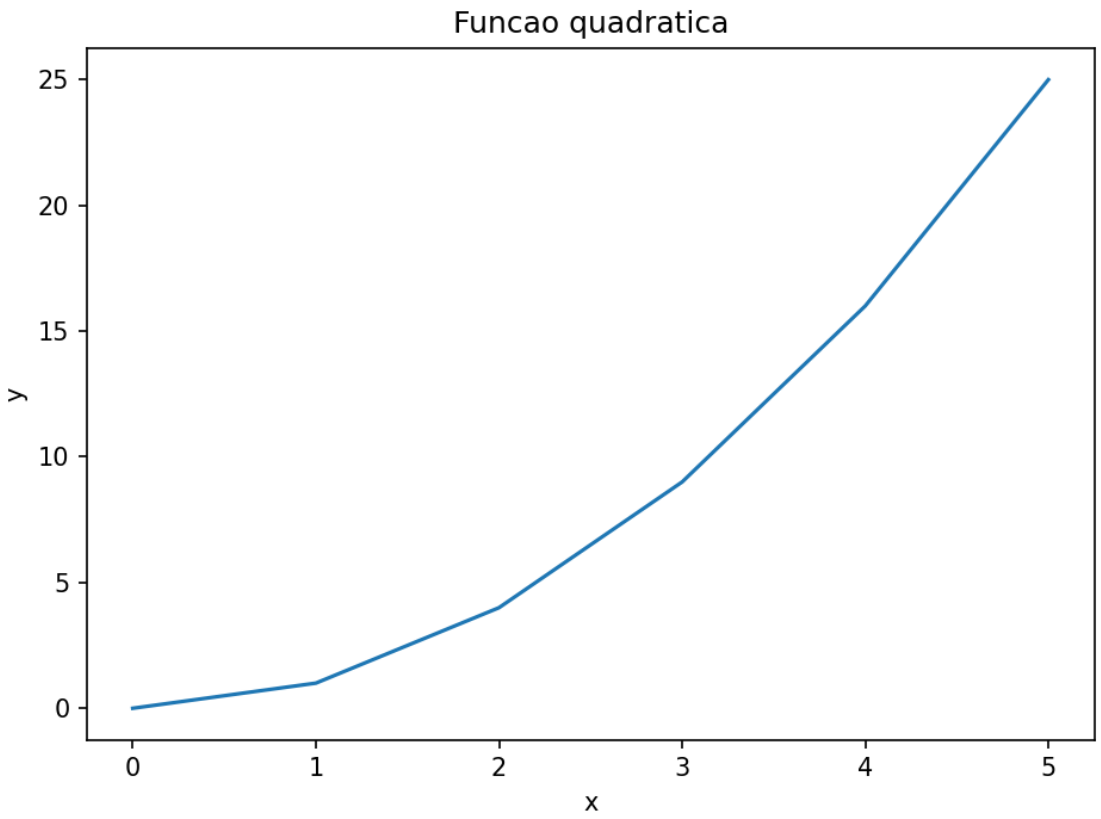
```
x = np.linspace(0, 5, 6)
y = x**2

plt.plot(x, y)
plt.title("Funcao quadratica")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Regra de ouro

Um gráfico sem título e sem rótulos raramente é informativo.

Resultado do gráfico de linha



Comparando Múltiplas Curvas

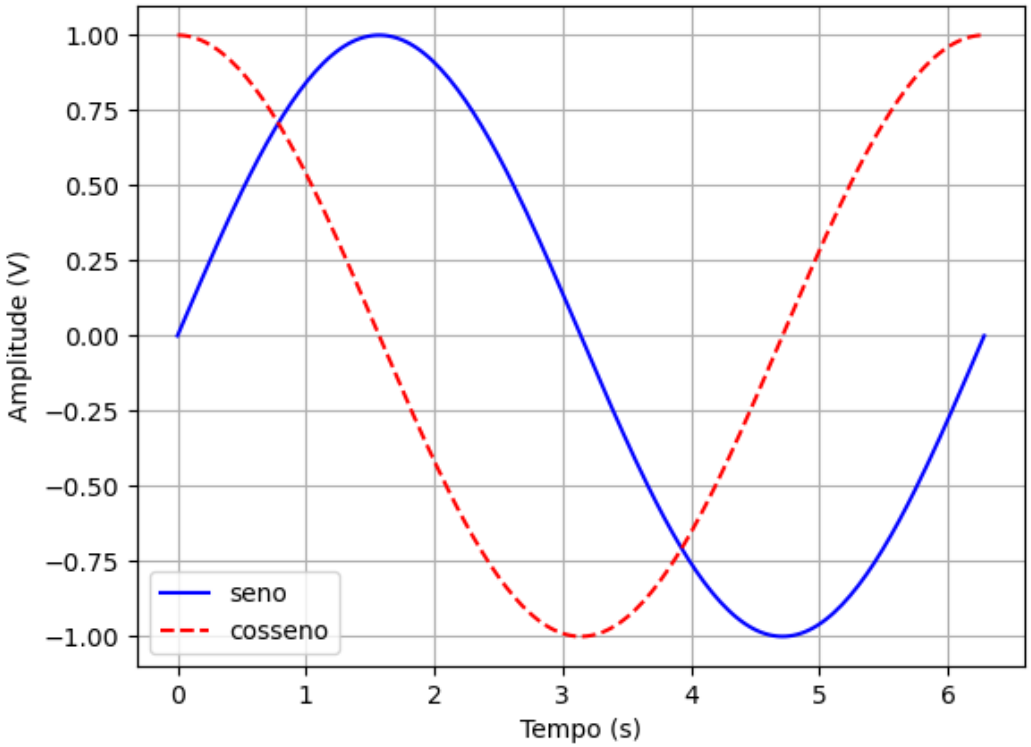
Dois `plot()` no mesmo eixo

```
x = np.linspace(0, 2*np.pi, 200)

plt.plot(x, np.sin(x), label='seno', color='blue')
plt.plot(x, np.cos(x), label='cosseno',
         linestyle='--', color='red')
plt.legend()
plt.grid(True)
plt.show()
```

- `label` define o texto da legenda.
- `grid(True)` ajuda a leitura visual dos valores.

Resultado da comparação de curvas



Customizações Básicas

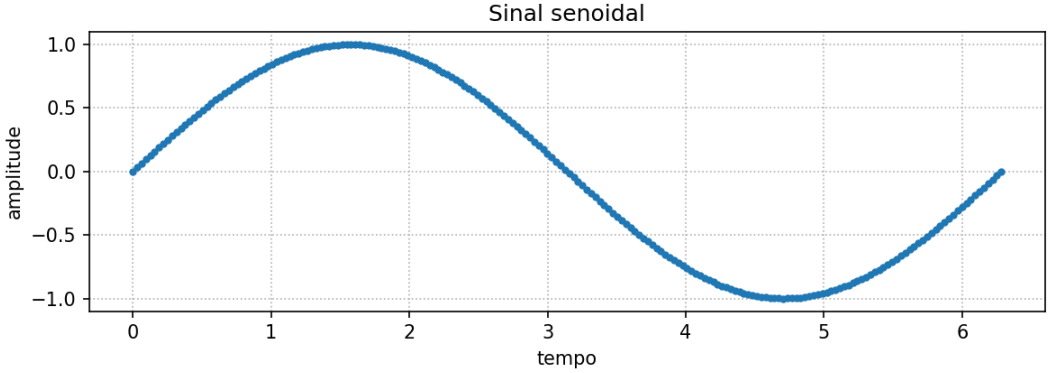
Controlando aparência

```
plt.figure(figsize=(8, 3))
plt.plot(x, np.sin(x), marker='o', markersize=3,
         linewidth=1.5)
plt.title("Sinal senoidal")
plt.xlabel("tempo")
plt.ylabel("amplitude")
plt.grid(True, linestyle=':')
```

Quando usar

Essas opções são úteis para tornar o gráfico mais legível, especialmente em relatórios e apresentações.

Resultado do gráfico customizado



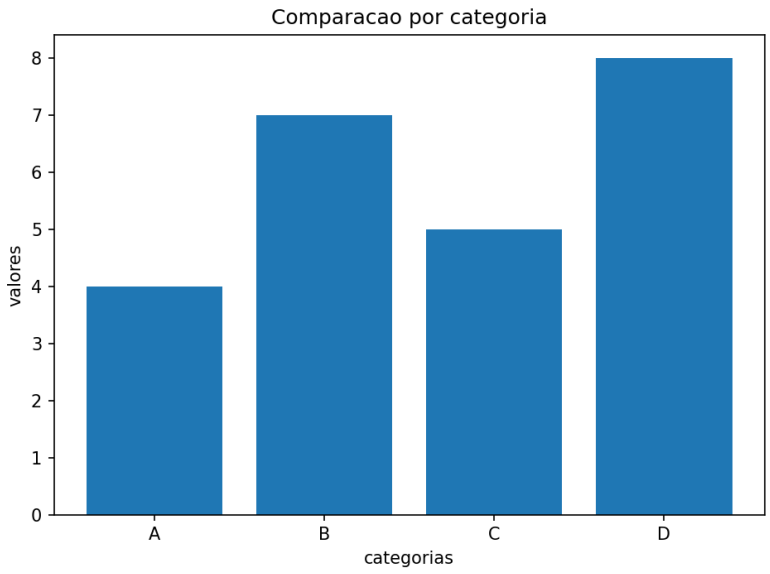
Outros Tipos de Gráfico

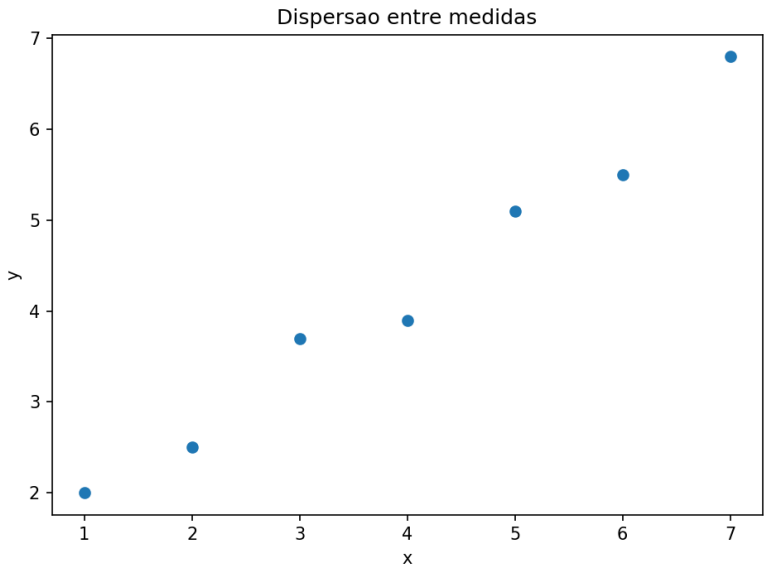
Barras, dispersão e histograma

```
plt.bar(categorias, valores)
plt.scatter(x_medidas, y_medidas)
plt.hist(amostras, bins=8, edgecolor='black')
```

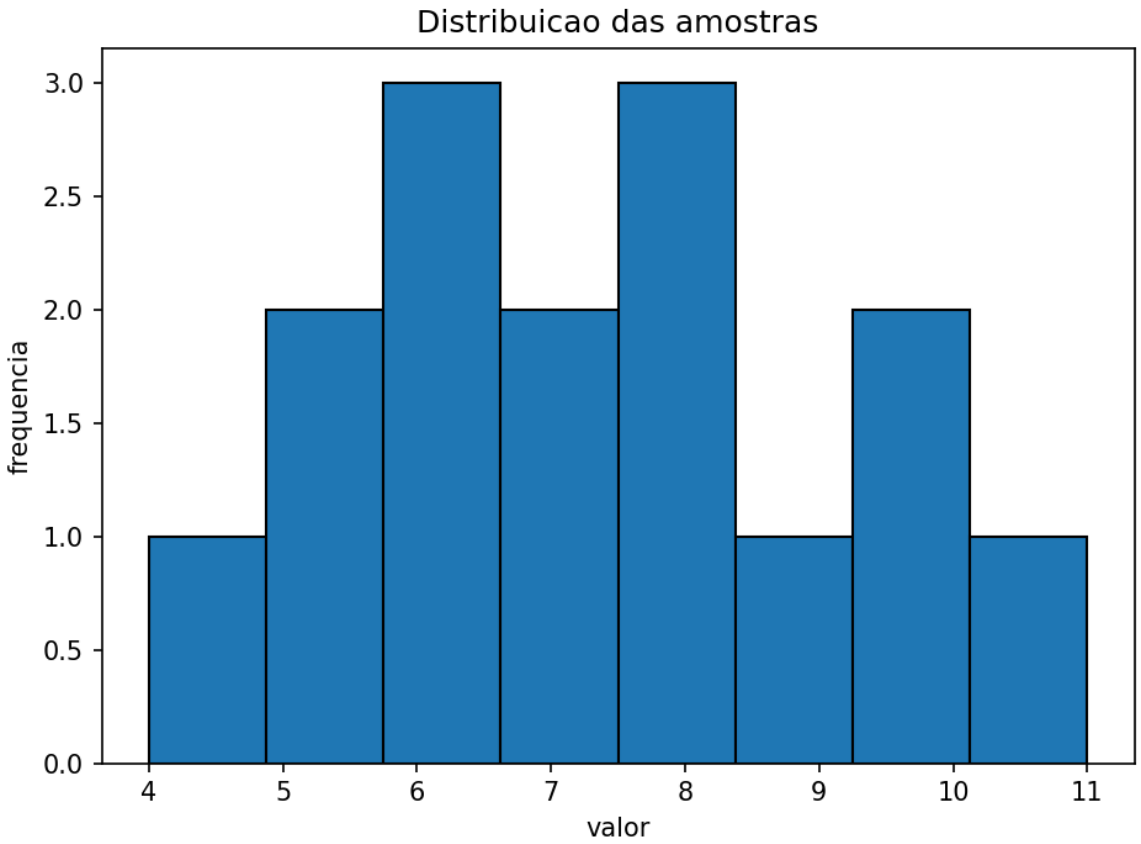
`bar`	Comparação entre categorias.
`scatter`	Relação entre duas variáveis numéricas.
`hist`	Distribuição de frequências.

Resultados: barras e dispersão





Resultado: histograma



Salvar Também Faz Parte do Fluxo

Ordem recomendada

```
plt.plot(x, y)
plt.title("Meu grafico")
plt.savefig("meu_grafico.png", dpi=150)
plt.show()
```

Pandas

Onde o Pandas entra nesse ecossistema?

- Quando os dados possuem colunas nomeadas, índices e mistura de tipos.
- Quando precisamos filtrar, agrupar e resumir informações tabulares.
- Quando queremos ler ou gravar dados em formatos simples, como CSV.

Series e DataFrame

Estruturas fundamentais

```
serie = pd.Series([7.5, 8.0, 6.5], index=['A', 'B', 'C'])

df = pd.DataFrame({
    'aluno': ['Ana', 'Bruno', 'Carla', 'Diego'],
    'turma': ['A', 'A', 'B', 'B'],
    'nota': [8.5, 7.0, 9.0, 6.5]
})
```

Interpretação

A `Series` é como uma coluna com rótulos. O `DataFrame` é uma tabela completa.

Seleção por Colunas e Linhas

Consultas básicas

```
print(df['nota'])
print(df.loc[0:1, ['aluno', 'nota']])
print(df.iloc[2:, 1:])
```

- `df['coluna']` seleciona uma coluna.
- `loc` usa rótulos.
- `iloc` usa posições numéricas.

Filtros e Novas Colunas

Seleção condicional

```
aprovados = df[df['nota'] >= 7.0]

df['situacao'] = np.where(df['nota'] >= 7.0,
                          'Aprovado',
                          'Recuperacao')
df['nota_ajustada'] = df['nota'] + 0.5
```

Ideia

Filtro booleano e criação de colunas novas aparecem em quase toda análise de dados.

Agregação, groupby e Ausentes

Resumo por turma

```
df.loc[3, 'nota'] = np.nan
df['nota'] = df['nota'].fillna(df['nota'].mean())

resumo = df.groupby('turma')['nota'].agg(['mean', 'max'])
print(resumo)
```

- `fillna()` trata ausências de forma simples.
- `groupby()` ajuda a transformar tabela em informação resumida.

Leitura e Escrita de CSV

Salvar e carregar um DataFrame com pandas

```
df.to_csv("dados.csv", index=False)

df_1 = pd.read_csv("dados.csv")

print(df_1.head())
```

O que o codigo faz

Primeiro, o DataFrame `df` e salvo no arquivo `dados.csv`. Depois, esse mesmo arquivo e lido novamente com pandas e armazenado em `df_1`.

Integração

Um Fluxo Pequeno e Completo

Da tabela ao gráfico

```
media_por_turma = df.groupby('turma')['nota'].mean()

plt.bar(media_por_turma.index, media_por_turma.values)
plt.title('Media de notas por turma')
plt.xlabel('Turma')
plt.ylabel('Media')
plt.show()
```

Mensagem final

NumPy ajuda a calcular, Pandas organiza e resume, Matplotlib comunica visualmente.

Resumo da Caixa de Ferramentas

Funções e ideias essenciais

- `np.array`, `np.linspace`, `np.arange`
- `shape`, slicing, operações vetorizadas, `np.mean`
- `plt.plot`, `plt.bar`, `plt.legend`, `plt.grid`
- `pd.Series`, `pd.DataFrame`, `loc`, filtros
- `groupby`, `agg`, `fillna`, `to_csv`

Exercícios Sugeridos

1. Crie um array com 60 pontos entre 0 e 10 representando o tempo em segundos. A partir dele, calcule duas curvas: uma tensão $v(t) = 12e^{-0.4t}$ e uma corrente $i(t) = 2(1 - e^{-0.4t})$.
2. Construa um gráfico comparando essas duas grandezas ao longo do tempo, com legenda, título e grade.
3. Monte um `DataFrame` com dados de equipamentos de um laboratório, incluindo colunas como `equipamento`, `setor`, `consumo_kwh` e `horas_uso`.
4. Filtre os equipamentos com consumo maior que 5 kWh e calcule o consumo médio por setor.
5. Gere um gráfico de barras com esse resumo para comparar o consumo médio entre os setores.